

SQL Case Study

Sales Order Processing System

A hands-on SQL assignment covering table design, constraints, SET operators, and multi-table joins — built around a retail case study for ABC Fashion.

- Table Creation & Data Insertion
- Constraints — Primary Key, Foreign Key, Default, NOT NULL
- Filtering with LIKE & WHERE conditions
- SET Operators — UNION vs UNION ALL
- Multi-table JOINS — INNER JOIN & RIGHT JOIN

Step 1: Problem Statement & Dataset — Salesman, Customer, and Orders tables

Problem Statement:

ABC Fashion is a leading retailer with a vast customer base and a team of dedicated sales representatives. They have a Sales Order Processing System that helps manage customer orders and interactions.

Dataset:

Find the below information and the script for the table creation and record insertion.

<https://drive.google.com/drive/folders/16s99CUGs3yZlpm60FOECb26h6aXrWr4?usp=sharing>

Salesman Table:

SalesmanId	SalesmanName	Commission	City	Age
101	Joe	50	California	17
102	Simon	75	Texas	25
103	Jessie	105	Florida	35
104	Danny	100	Texas	22
105	Lia	65	New Jersey	30

Customer Table:

SalesmanId	CustomerId	CustomerName	PurchaseAmount
101	2345	Andrew	550
103	1575	Lucky	4500
104	2345	Andrew	4000
107	3747	Remona	2700
110	4004	Julia	4545

Orders Table:

OrderId	CustomerId	SalesmanId	OrderDate	Amount
5001	2345	101	04-07-2021	550
5003	1234	105	15-02-2022	1500

Step 2: Tasks to be Performed

Tasks to be Performed:

1. Insert a new record in your Orders table.
2. Add Primary key constraint for SalesmanId column in Salesman table. Add default constraint for City column in Salesman table. Add Foreign key constraint for SalesmanId column in Customer table. Add not null constraint in Customer_name column for the Customer table.
3. Fetch the data where the Customer's name is ending with 'N' also get the purchase amount value greater than 500.
4. Using SET operators, retrieve the first result with unique SalesmanId values from two tables, and the other result containing SalesmanId with duplicates from two tables.
5. Display the below columns which has the matching data.
Orderdate, Salesman Name, Customer Name, Commission, and City which has the range of Purchase Amount between 500 to 1500.
6. Using right join fetch all the results from Salesman and Orders table.

Step 3: Table Creation & Record Insertion — Salesman, Customer tables

V\DELL (85) ✕

--Tasks to be Performed:

--Salesman table creation

```
CREATE TABLE Salesman (  
SalesmanId INT,  
Name VARCHAR(255),  
Commission DECIMAL(10, 2),  
City VARCHAR(255),  
Age INT  
);
```

--Salesman table record insertion

```
INSERT INTO Salesman (SalesmanId, Name, Commission, City, Age)  
VALUES  
(101, 'Joe', 50, 'California', 17),  
(102, 'Simon', 75, 'Texas', 25),  
(103, 'Jessie', 105, 'Florida', 35),  
(104, 'Danny', 100, 'Texas', 22),  
(105, 'Lia', 65, 'New Jersey', 30);
```

--Customer Table

--Customer table creation

```
CREATE TABLE Customer (  
SalesmanId INT,  
CustomerId INT,  
CustomerName VARCHAR(255),  
PurchaseAmount INT,
```

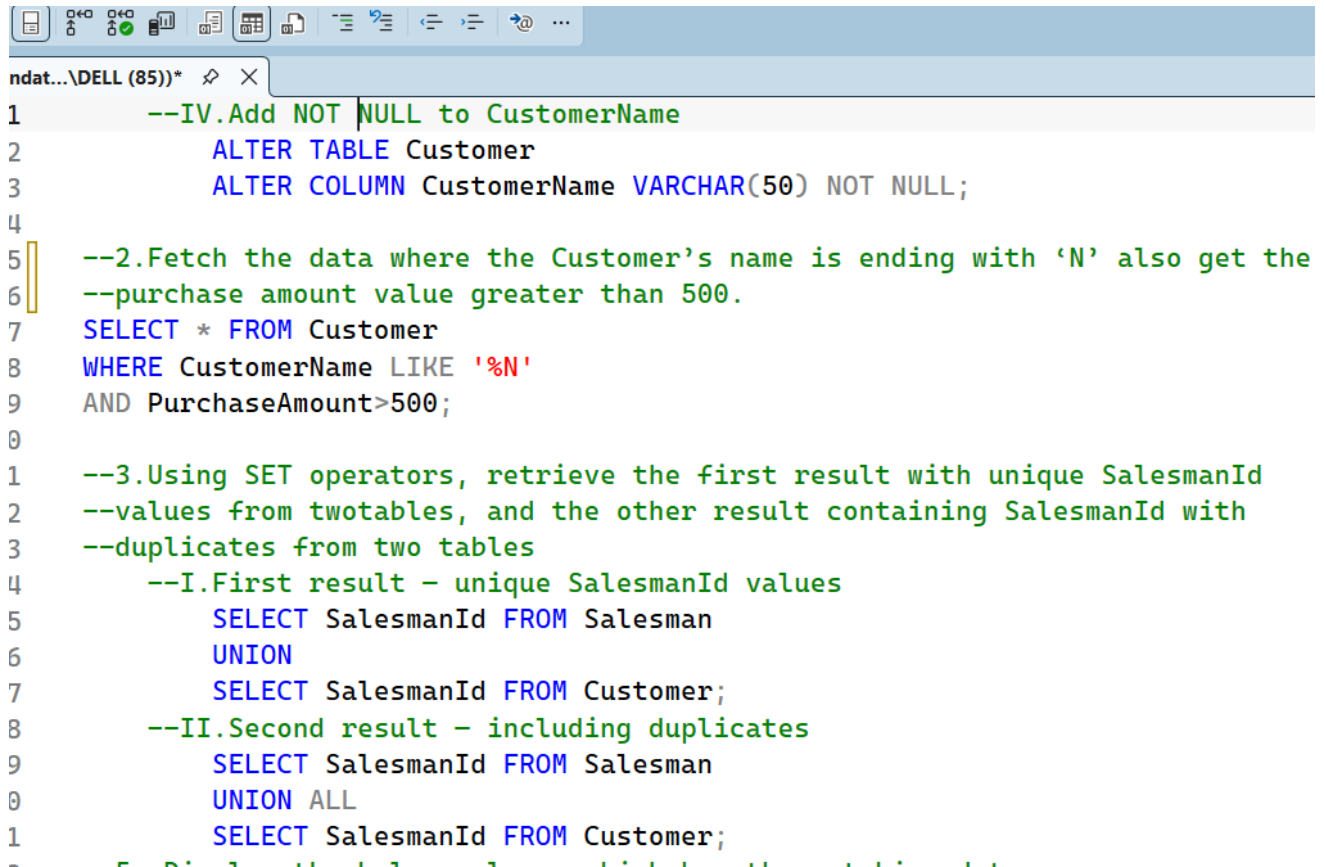
Step 4: Table Creation & Record Insertion — Orders table + sample SELECTs

```
landat...\DELL (85))* ×
28      --Customer table record insertion
29      INSERT INTO Customer (SalesmanId, CustomerId, CustomerName, PurchaseAmount)
30      VALUES
31      (101, 2345, 'Andrew', 550),
32      (103, 1575, 'Lucky', 4500),
33      (104, 2345, 'Andrew', 4000),
34      (107, 3747, 'Remona', 2700),
35      (110, 4004, 'Julia', 4545);
36
37      --Orders Table
38      --Orders table Creation
39      CREATE TABLE Orders (OrderId int, CustomerId int, SalesmanId int, Orderdate Date,
40      Amount money)
41
42      --Orders table record insertion
43      INSERT INTO Orders Values
44      (5001,2345,101, '2021-07-01', 550),
45      (5003,1234,105, '2022-02-15', 1500)
46
47      select * from Salesman
48      select * from Customer
49      select * from Orders
50
```

Step 5: Adding Constraints — Primary Key, Default, Foreign Key

```
SQL Mandat...DELL (85))* X
56 --2. Add Primary key constraint for SalesmanId column in Salesman table.
57 --Add default constraint for City column in Salesman table.
58 --Add Foreign key constraint for SalesmanId column in Customer table.
59 --Add not null constraint in Customer_name column for the Customer table.
60
61 --I.Add Primary Key to SalesmanId
62     ALTER TABLE Salesman
63     ALTER COLUMN SalesmanId INT NOT NULL;
64     ALTER TABLE Salesman
65     ADD CONSTRAINT PK_Salesman_SalesmanId
66     PRIMARY KEY (SalesmanId);
67 --II.Add Default Constraint to City
68     ALTER TABLE Salesman
69     ADD CONSTRAINT DF_Salesman_City
70     DEFAULT 'Unknown' FOR City;
71 --III.Add Foreign Key to SalesmanId in Customer
72     UPDATE Customer
73     SET SalesmanId = 101
74     WHERE SalesmanId IN (107, 110);
75
76     ALTER TABLE Customer
77     ADD CONSTRAINT FK_Customer_SalesmanId
78     FOREIGN KEY (SalesmanId)
79     REFERENCES Salesman(SalesmanId);
80     SELECT * FROM Customer
81 --IV.Add NOT NULL to CustomerName
```

Step 6: NOT NULL Constraint, Filtering with LIKE, and SET Operators (UNION / UNION ALL)



```
ndat...\DELL (85))*
1      --IV.Add NOT NULL to CustomerName
2      ALTER TABLE Customer
3      ALTER COLUMN CustomerName VARCHAR(50) NOT NULL;
4
5      --2.Fetch the data where the Customer's name is ending with 'N' also get the
6      --purchase amount value greater than 500.
7      SELECT * FROM Customer
8      WHERE CustomerName LIKE '%N'
9      AND PurchaseAmount>500;
10
11     --3.Using SET operators, retrieve the first result with unique SalesmanId
12     --values from twotables, and the other result containing SalesmanId with
13     --duplicates from two tables
14     --I.First result - unique SalesmanId values
15     SELECT SalesmanId FROM Salesman
16     UNION
17     SELECT SalesmanId FROM Customer;
18     --II.Second result - including duplicates
19     SELECT SalesmanId FROM Salesman
20     UNION ALL
21     SELECT SalesmanId FROM Customer;
```

Step 7: Multi-table JOINS — INNER JOIN and RIGHT JOIN queries

```
Mandat...\DELL (85))* X
L02  --5..Display the below columns which has the matching data.
L03  --Orderdate, Salesman Name, Customer Name, Commission, and
L04  --City which has therange of Purchase Amount between 500 to 1500
L05  SELECT
L06      O.OrderDate,
L07      S.Name,
L08      C.CustomerName,
L09      S.Commission,
L10      S.City
L11  FROM Orders O
L12  INNER JOIN Salesman S
L13      ON O.SalesmanId = S.SalesmanId
L14  INNER JOIN Customer C
L15      ON O.CustomerId = C.CustomerId
L16      AND O.SalesmanId = C.SalesmanId
L17  WHERE C.PurchaseAmount BETWEEN 500 AND 1500;
L18  --6. Using right join fetch all the results from Salesman and Orders table.
L19  SELECT
L20      S.SalesmanId,
L21      S.Name,
L22      S.Commission,
L23      S.City,
L24      S.Age,
L25      O.OrderId,
L26      O.CustomerId,
L27      O.OrderDate,
L28      O.Amount
L29  from Salesman S
L30  RIGHT JOIN Orders O
L31      ON S.SalesmanId=O.SalesmanId;
```